

LAB TASK#10

NAME : MASHAAL ISHTIAQ

ROLL NO: 22P-9326

```
#include <iostream>
using namespace std;

class Node
{

private:
    int val;
    Node *left;
    Node *right;

public:
    Node()
    { // default constructor
        this->val = 0;
        left = right = NULL;
    }

    Node(int val)
    { // parameterized constructor
        this->val = val;
        left = right = NULL;
    }

    friend class BST;
    friend int main();
};

class BST
{

private:
    Node *root;

public:
    BST()
    {
        root = NULL;
    }

    Node *insert(Node *node, int val)
    {

        if (node == NULL)
        {
            Node *temp = new Node(val);
            if (root == NULL)
            {
                root = temp;
            }
            return temp;
        }

        else
        {
            if (node->val == val)
            {
                cout << "Already Exist" << endl;
                return node;
            }
            else if (val < node->val)
```

```
            node->left = insert(node->left, val);
        else
        {
            node->right = insert(node->right, val);
        }
        return node;
    }
}

//_____to find minimum node on the subtree ->as we will move towards the
right due to the deletion of a node having two children

Node *MinNode(Node *node)
{

    while (node->left != NULL)

        node = node->left;

    return node;
}

//____deleting the node

Node *DeleteNode(Node *node, int val)
{

    if (node == NULL)

        return NULL; // indicates that the tree is empty

    //__searching the node and deleting the node by recurssive ftn
    if (val < node->val)

        node->left = DeleteNode(node->left, val);

    else if (val > node->val)

        node->right = DeleteNode(node->right, val);

    else
    {

        //___if the node to be deleted has no child
        if (node->left == NULL && node->right == NULL)
        {
            delete node;
            return NULL;
        }

        //___if the node to be deleted has one child

        else if (node->left == NULL)
        {
            Node *temp = node->right;

            delete node;

            return temp;
        }

        else if (node->right == NULL)
        {
            Node *temp = node->left;
```

```
                delete node;

                return temp;
            }

            else
            {
                Node *temp = MinNode(node->right);

                node->val = temp->val;

                node->right = DeleteNode(node->right, temp->val);
            }
                }
        return node;
    }

    int PreOrderTraversal(Node *temp)
    { // here temp is root node

        if (temp == NULL)
            return 0;



        cout << temp->val << "\t";

         PreOrderTraversal(temp->left);

        PreOrderTraversal(temp->right);

        return 0;
    }
    friend int main();
};

int main()
{
    BST bst;
    // hard code
    bst.insert(bst.root, 50);
    bst.insert(bst.root, 30);
    bst.insert(bst.root, 70);
    bst.insert(bst.root, 20);
    bst.insert(bst.root, 40);
    bst.insert(bst.root, 60);
    bst.insert(bst.root, 80);

    cout << "Before deletion: ";

    bst.PreOrderTraversal(bst.root);

    cout << endl;

    int val = 50;

    bst.DeleteNode(bst.root, val);

    cout << "Node with value " << val << " deleted!" << endl;

    cout << "After deletion: ";

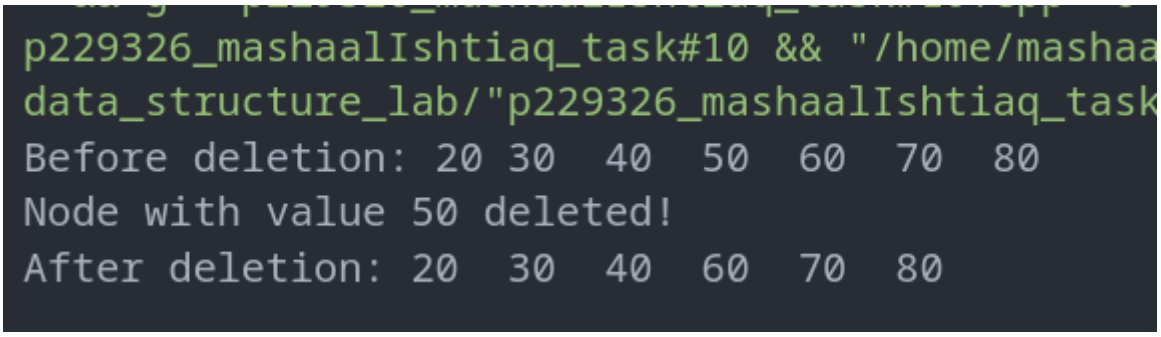
    bst.PreOrderTraversal(bst.root);
```

```
    cout << endl;

    return 0;
}
```

NOTE:

If in-order is used then the output will be like this as in in-order it prints in descending order



If pre-order is used then the output will be like this

